

Core Language Working Group "ready" Issues for the March, 2026 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N5032](#).

2609. Padding in class types

Section: 7.6.2.5 [\[expr.sizeof\]](#) **Status:** ready **Submitter:** Jim X **Date:** 2022-07-19

Class types may have padding, influencing the result of `sizeof`. It is unclear whether the placement and amount of padding is implementation-defined, unspecified, or something else. If it is unspecified, the limits of permissible behavior are unclear. Empty classes might need special consideration.

Proposed resolution (approved by CWG 2026-02-20):

1. Add a new entry in Clause 3 [\[intro.defs\]](#):

3.nn [\[defns.impl.prop\]](#)

property of the implementation

behavior, for a well-formed program (3.68 [\[defns.well.formed\]](#)) construct and correct data, that depends on the implementation

2. Change in 4.1.2 [\[intro.abstract\]](#) paragraph 2 as follows:

Certain aspects and operations of the abstract machine **constitute the parameters of the abstract machine and** are described in this document as implementation-defined behavior (for example, `sizeof(int)`) **or as properties of the implementation (for example, padding in class types)**. ~~These constitute the parameters of the abstract machine. Each~~ **For implementation-defined behavior, each** implementation shall include documentation describing its characteristics and behavior in these respects. Such documentation shall define the instance of the abstract machine that corresponds to that implementation (referred to as the “corresponding instance” below)

3. Change in 7.6.2.5 [\[expr.sizeof\]](#) paragraph 2

... When applied to a class, the result is the number of bytes in an object of that class including any padding required for placing objects of that type in an array. **The amount and placement of padding in a class type is a property of the implementation.** The result of applying `sizeof` to a potentially-overlapping subobject is the size of the type, not the size of the subobject. [Footnote: ...]

2660. Confusing term "this parameter"

Section: 7.6.1.3 [\[expr.call\]](#) **Status:** ready **Submitter:** Anoop Rana **Date:** 2022-11-23

Subclause 7.6.1.3 [\[expr.call\]](#) paragraph 7 specifies:

If the function is an implicit object member function, the `this` parameter of the function (7.5.3 [\[expr.prim.this\]](#)) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion (7.6.3 [\[expr.cast\]](#)).

The term "this parameter" is undefined.

Proposed resolution (approved by CWG 2026-03-27):

Change in 7.6.1.3 [\[expr.call\]](#) paragraph 7 as follows:

If the function is an implicit object member function, the **this implicit object** parameter of the function (7.5.3 [\[expr.prim.this\]](#)) is initialized with a pointer to the object of the call, converted as if by an explicit type conversion (7.6.3 [\[expr.cast\]](#)).

2744. Multiple objects of the same type at the same address

Section: 6.8.2 [\[intro.object\]](#) **Status:** ready **Submitter:** Chris Hallock **Date:** 2023-06-08

(From thread beginning [here](#).)

Consider:

```
#include <new>

struct A { unsigned char buf[1]; };
```

```
static_assert(sizeof(A) == 1); // A can fit within A::buf

int main()
{
    A x{};
    new (x.buf) A{};
}
```

`A::buf` provides storage for another `A` object. Thus, there are now two objects of type `A` within lifetime, which is inconsistent with the goal expressed by 6.8.2 [intro.object] paragraph 9.

Suggested resolution [SUPERSEDED]:

Change in 6.8.2 [intro.object] paragraph 3 as follows:

If a complete object **of type `T`** is created (7.6.2.8 [expr.new]) in storage associated with another object `e` of type “array of `N` unsigned char” or of type “array of `N` `std::byte`” (17.2.1 [cstddef.syn]), that array provides storage for the created object if:

- the lifetime of `e` has begun and not ended, and
- the storage for the new object fits entirely within `e`, and
- **`e` is not and is not nested within an object of type similar (7.3.6 [conv.qual]) to `T` that is within its lifetime, and**
- there is no array object that satisfies these constraints nested within `e`.

Additional notes (CWG 2026-01-23):

Consider:

```
A x{}: // implicit object creation for x.buf
A* p = reinterpret_cast<A*>(x.buf); // nailed down A at x.buf
```

There is no pointer interconvertibility between `x.buf` and `x`, so there must be a second `A` at `x.buf`.

Also consider:

```
struct S {
    unsigned char c[1];
};
unsigned char a[sizeof(S)];
S* ps = new (a) S; // must create S inside a buffer; now we have two "array of unsigned char" at the same address
```

Also consider:

```
struct B {};
struct A : B {};
struct C : B {};
struct D : A, C { unsigned char x[2]; };

D d;
new (d.x+1) C();
```

The `A` and `C` base classes must have different offsets (because their `B` bases must have different offsets), thus `#2` creates a `C` object at the offset of the existing `C` base class object.

CWG leaned towards adding a targeted exception that two objects of the same type may be located at the same address if both are nested within the same complete object.

Proposed resolution (approved by CWG 2026-02-06):

Change in 6.8.2 [intro.object] paragraph 10 as follows:

... Two objects with overlapping lifetimes that are not bit-fields may have the same address if

- one is nested within the other,
- they are both nested within some complete object *o*, exactly one is a subobject of *o*, and the subobject is of zero size,
- **they are both subobjects of the same complete object**, at least one is a subobject of zero size, and they are not of similar types (7.3.6 [conv.qual]),
- or
- they are both potentially non-unique objects;

otherwise, they have distinct addresses and occupy disjoint bytes of storage. [Footnote: ...]

2765. Address comparisons between potentially non-unique objects during constant evaluation

Section: 6.8.2 [intro.object] **Status:** ready **Submitter:** CWG **Date:** 2023-07-14

The (arguably) expanded treatment of backing arrays and string literals as potentially non-unique objects in issue 2753 lead to the question how the resulting address comparisons are treated during constant evaluation.

Subclause 7.7 [expr.const] bullet 5.24 specifies:

An expression `E` is a *core constant expression* unless the evaluation of `E`, following the rules of the abstract machine (6.10.1 [intro.execution]), would evaluate one of the following:

- ...

- a three-way comparison (7.6.8 [expr.spaceship]), relational (7.6.9 [expr.rel]), or equality (7.6.10 [expr.eq]) operator where the result is unspecified;
- ...

This phrasing is understood to refer to explicitly unspecified outcomes only. The treatment of an example such as

```
constexpr bool b = "abc" == "abc";
```

is unclear, given that identical string literals may or may not yield distinct string literal objects.

The assumption that equality comparison of `std::string_view` would compare addresses as a short-cut before comparing the character sequence could not be confirmed (27.2.2 [char.traits.require], 27.3.3.8 [string.view.ops] paragraph 12).

CWG in Tokyo 2024-03-22

Different approaches are feasible:

- Require merging of string literals (and backing arrays of character types?) at compile-time: Comparing addresses of string literals is equivalent to comparing their contents.
- Being able to guarantee that addresses of string literals with different contents are unequal also at compile time seems useful. In particular, this idea would allow comparisons during constant evaluation if (and only if) the runtime result is reliable.
- At compile time, tag each evaluation of a string literal with a globally-unique counter value. Comparing two such pointers with different tags makes the evaluation not a constant expressions.
- At compile time, each string literal (as written in the source code) yields a different address. Repeated evaluations of the same string literal yield pointers that compare equal. This can be achieved by tagging the pointer to the string literal obtained upon evaluation with the source location of the string literal.

In the latter cases, tag values can be preserved when performing pointer arithmetic.

Possible resolution (January, 2025) [SUPERSEDED]

1. Add a new paragraph before 6.9.4 [basic.compound] paragraph 4:

A pointer value pointing to a potentially non-unique object *O* (6.8.2 [intro.object]) is *associated with* the evaluation of the *string-literal* (5.13.5 [lex.string]) or initializer list (9.5.5 [decl.init.list]) that resulted in the string literal object or backing array, respectively, that is *O* or of which *O* is a subobject. [Note: A pointer value obtained by pointer arithmetic (7.6.6 [expr.add]) from a pointer value associated with an evaluation *E* is also associated with *E*. -- end note]

A pointer value *P* is valid in the context of an evaluation *E* if ...

2. Add a bullet after 7.7 [expr.const] bullet 10.25 as follows:

An expression *E* is a core constant expression unless the evaluation of *E*, following the rules of the abstract machine (6.10.1 [intro.execution]), would evaluate one of the following:

- ...
- a three-way comparison (7.6.8 [expr.spaceship]), relational (7.6.9 [expr.rel]), or equality (7.6.10 [expr.eq]) operator where the result is unspecified;
- an equality operator comparing pointers to potentially non-unique objects, if the pointer values of both operands are associated with different evaluations (6.9.4 [basic.compound]) and they can both point to the same offset within the same potentially non-unique object; [Example:

```
constexpr const char *f() { return "foo"; }

constexpr bool b1 = "foo" == "foo";    // error: non-constant
constexpr bool b2 = f() == f();        // error: non-constant
constexpr const char *p = f();
constexpr bool b3 = p == p;            // OK, value of b3 is true
constexpr bool b4 = "xfoo" + 1 == "foo\0y"; // error: non-constant; string literal object could contain "xfoo\0y"
constexpr bool b5 = "foo" == "bar";    // OK, value of b5 is false
constexpr bool b6 = "foo" == "oo";     // OK, value of b6 is false; offsets would be different in a merged string literal object
```

-- end example]

- ...

Proposed resolution (February 2026) [SUPERSEDED]:

1. Add a new paragraph before 6.9.4 [basic.compound] paragraph 4:

A pointer value pointing to a potentially non-unique object *O* (6.8.2 [intro.object]) is *associated with* the evaluation of the *string-literal* (5.13.5 [lex.string]) or initializer list (9.5.5 [decl.init.list]) that resulted in the string literal object or backing array, respectively, that is *O* or of which *O* is a subobject. [Note: A pointer value obtained by pointer arithmetic (7.6.6 [expr.add]) from a pointer value associated with an evaluation *E* is also associated with *E*. -- end note]

A pointer value *P* is valid in the context of an evaluation *E* if ...

2. Add a paragraph before 7.7 [expr.const] paragraph 9 as follows:

A type has *constexpr-unknown representation* if it

- is a union,
- is a pointer or pointer-to-member type,
- is volatile-qualified,
- is a class type with a non-static data member of reference type, or
- has a base class or a non-static member whose type has *constexpr-unknown representation*.

3. Add a bullet after 7.7 [expr.const] bullet 9.25 as follows:

An expression E is a core constant expression unless the evaluation of E, following the rules of the abstract machine (6.10.1 [intro.execution]), would evaluate one of the following:

- o ...
- o a three-way comparison (7.6.8 [expr.spaceship]), relational (7.6.9 [expr.rel]), or equality (7.6.10 [expr.eq]) operator where the result is unspecified;
- o an equality operator comparing pointers to potentially non-unique objects, if the pointer values of both operands are associated with different evaluations (6.9.4 [basic.compound]) and either they can both point to the same offset within the same potentially non-unique object or one of them points to an object whose type has constexpr-unknown representation; [Example:

```
constexpr const char *f() { return "foo"; }

constexpr bool b1 = "foo" == "foo";    // error: non-constant
constexpr bool b2 = f() == f();        // error: non-constant
constexpr const char *p = f();
constexpr bool b3 = p == p;            // OK, value of b3 is true
constexpr bool b4 = "xfoo" + 1 == "foo\0y"; // error: non-constant; string literal object could contain "xfoo\0y"
constexpr bool b5 = "foo" == "bar";    // OK, value of b5 is false
constexpr bool b6 = "foo" == "oo";     // OK, value of b6 is false; offsets would be different in a merged string literal object

-- end example ]
```

- o ...

4. Change in 22.11.3 [bit.cast] paragraph 3 as follows:

Constant When: To **and** From **do not have constexpr-unknown representation** and the types of all subobjects of To and From are types T such that:

- o is_union_v<T> is false;
- o is_pointer_v<T> is false;
- o is_member_pointer_v<T> is false;
- o is_volatile_v<T> is false; and
- o T has no non-static data members of reference type.

CWG 2026-02-20

A template parameter object of array type is also a potentially non-unique object (see issue 3141) and needs to be handled by the wording.

Proposed resolution (approved by CWG 2026-03-24; approved by LWG 2026-03-23):

1. Add a new paragraph before 6.9.4 [basic.compound] paragraph 4:

A pointer value pointing to a potentially non-unique object O (6.8.2 [intro.object]) is associated with the evaluation of

- o the string-literal (5.13.5 [lex.string]) that resulted in the string literal object,
- o the initializer list (9.5.5 [dcl.init.list]) that resulted in the backing array, or
- o the initialization of the template parameter object (13.4.3 [temp.arg.nontype], 21.4.3 [meta.define.static])

that is O or of which O is a subobject. [Note: A pointer value obtained by pointer arithmetic (7.6.6 [expr.add]) from a pointer value associated with an evaluation E is also associated with E. -- end note]

A pointer value P is valid in the context of an evaluation E if ...

2. Add a paragraph before 7.7 [expr.const] paragraph 9 as follows:

A type has *constexpr-unknown representation* if it

- o is a union,
- o is a pointer or pointer-to-member type,
- o is volatile-qualified,
- o is a class type with a non-static data member of reference type, or
- o has a base class or a non-static member whose type has constexpr-unknown representation.

3. Add a bullet after 7.7 [expr.const] bullet 9.25 as follows:

An expression E is a core constant expression unless the evaluation of E, following the rules of the abstract machine (6.10.1 [intro.execution]), would evaluate one of the following:

- o ...
- o a three-way comparison (7.6.8 [expr.spaceship]), relational (7.6.9 [expr.rel]), or equality (7.6.10 [expr.eq]) operator where the result is unspecified;
- o an equality operator comparing pointers to potentially non-unique objects, if the pointer values of the two operands are associated with different evaluations (6.9.4 [basic.compound]) and either they can both point to the same offset within the same potentially non-unique object or one of them points to an object whose type has constexpr-unknown representation; [Example:

```
constexpr const char *f() { return "foo"; }

constexpr bool b1 = +"foo" == "foo";    // error: non-constant
constexpr bool b2 = f() == f();        // error: non-constant
constexpr const char *p = f();
constexpr bool b3 = p == p;            // OK, value of b3 is true
constexpr bool b4 = "xfoo" + 1 == "foo\0y"; // error: non-constant; string literal object could contain "xfoo\0y"
constexpr bool b5 = "foo" == "bar" + 0;    // OK, value of b5 is false
constexpr bool b6 = (const char*)"foo" == "oo"; // OK, value of b6 is false; offsets would be different in a merged string literal object

constexpr std::initializer_list<int *> il1 = { (int *)nullptr };
constexpr std::initializer_list<unsigned long> il2 = { 0 };
constexpr bool b7 = il1.begin() == (void *)il2.begin(); // error: non-constant
```

-- end example]

o ...

4. Change in 22.11.3 [bit.cast] paragraph 3 as follows:

Constant When: Neither To nor From has constexpr-unknown representation (7.7 [expr.const]), and the types of all subobjects of To and From are types T such that:

- o is_union_v<T> is false;
- o is_pointer_v<T> is false;
- o is_member_pointer_v<T> is false;
- o is_volatile_v<T> is false; and
- o T has no non-static data members of reference type.

2799. Inheriting default constructors

Section: 11.4.5.2 [class.default.ctor] **Status:** ready **Submitter:** Hubert Tong **Date:** 2017-09-01

(See also submission #545.)

Consider:

```
struct A { int n; };
struct B : A {
    using A::A;
    B(int);
};
```

Does B have a default constructor?

Suggested resolution [SUPERSEDED]:

1. Change in 9.10 [namespace.udecl] paragraph 4 as follows:

If a constructor or assignment operator brought from a base class into a derived class has the signature of a **default constructor or** copy/move constructor or assignment operator for the derived class (11.4.5.2 [class.default.ctor], 11.4.5.3 [class.copy.ctor], 11.4.6 [class.copy.assign]), the *using-declaration* does not by itself suppress the implicit declaration of the derived class member; the member from the base class is hidden or overridden by the implicitly-declared ~~copy/move constructor or assignment operator~~ **special member function** of the derived class, as described below.

2. Change in 11.4.5.2 [class.default.ctor] paragraph 1 as follows:

A *default constructor* for a class X is a constructor of class X for which each parameter that is not a function parameter pack has a default argument (including the case of a constructor with no parameters). If there is no user-declared constructor for class X, **or if X inherits (9.10 [namespace.udecl]) one or more default constructors and there is no user-declared default constructor for X**, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.6 [dcl.fct.def]). An implicitly-declared default constructor is an inline public member of its class.

Proposed resolution (2023-10) [SUPERSEDED]:

(This also resolves issue 2632.)

1. Change in 9.10 [namespace.udecl] paragraph 4 as follows:

~~If a constructor or assignment operator brought from a base class into a derived class has the signature of a copy/move constructor or assignment operator for the derived class (11.4.5.3 [class.copy.ctor], 11.4.6 [class.copy.assign]), the *using-declaration* does not by itself suppress the implicit declaration of the derived class member; the member from the base class is hidden or overridden by the implicitly-declared copy/move constructor or assignment operator of the derived class, as described below. [Note: A *using-declaration* that names a member function of a base class does not suppress the implicit declaration of a special member function in the derived class, even if their signatures are the same (11.4.5.2 [class.default.ctor], 11.4.5.3 [class.copy.ctor], 11.4.6 [class.copy.assign]). -- end note]~~

2. Add a new paragraph before 11.4.1 [class.mem.general] paragraph 2 as follows:

... For any other *member-declaration*, each declared entity that is not an unnamed bit-field (11.4.10 [class.bit]) is a member of the class, **termed a user-declared member**, and each such *member-declaration* shall either declare at least one member name of the class or declare at least one unnamed bit-field.

3. Change in 11.4.5.2 [class.default.ctor] paragraph 1 as follows:

A *default constructor* for a class X is a constructor of class X for which each parameter that is not a function parameter pack has a default argument (including the case of a constructor with no parameters). If ~~there is no~~ **a class X does not have a user-declared constructor for class X, or if X inherits (9.10 [namespace.udecl]) one or more default constructors and X does not have a user-declared default constructor**, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.6 [dcl.fct.def]). An implicitly-declared default constructor is an inline public member of its class. **[Example:**

```
struct A {};
struct B {};
struct C : A, B {
    using A::A, B::B;
    C(int);
};
C c;    // OK
```

```

struct X { X(int = 0, int = 0); }; // #1
struct Y { Y(int = 0); }; // #2
struct Z : X, Y {
    using X::X, Y::Y;
};
Z z2(1, 1); // OK, invokes X(1, 1) and Y()
Z z1(1); // error: ambiguous between #1 and #2
Z z0; // OK, invokes X() and Y()

```

-- end example]

4. Change in 11.4.5.3 [class.copy.ctor] paragraph 6 as follows:

If the class **definition** does not **explicitly declare** **have** a **user-declared** copy constructor, a non-explicit one is declared implicitly. ...

5. Change in 11.4.5.3 [class.copy.ctor] paragraph 8 as follows:

If **the definition of** a class X does not **explicitly declare** **have** a **user-declared** move constructor, a non-explicit one will be implicitly declared as defaulted if and only if ...

6. Add a new paragraph before 11.4.5.3 [class.copy.ctor] paragraph 11 as follows:

[Note: A *using-declaration* in a derived class C that names a constructor from a base class never suppresses the implicit declaration of a copy/move constructor of C, even if the base class constructor would be a copy or move constructor if declared as a member of C. -- end note]

A copy/move constructor for class X is trivial if it is not user-provided and if: ...

7. Change in 11.4.6 [class.copy.assign] paragraph 2 as follows:

If the class **definition** does not **explicitly declare** **have** a **user-declared** copy assignment operator, one is declared implicitly. If the class **definition** **declares** **has** a **user-declared** move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defaulted (9.6 [dcl.fct.def]). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor (D.6 [depr.impldec]). ...

Additional notes (November, 2023)

How does access checking interact with the proposed resolution above?

```

struct B {
protected:
    B(int = 0);
};
struct A : B {
    using B::B;
    A(void *);
};
A a; // okay?
A aa(42); // not okay

```

CWG 2023-11-06

CWG resolved not to declare a default constructor in the derived class, but instead apply the usual rules for inherited constructors for this case. The wording should be changed so that the presence of a default constructor is never checked, in particular for "trivial class" (11.2 [class.prop], fixed by P3247R1 (Deprecate the notion of trivial types)), vacuous initialization (6.8.4 [basic.life] paragraph 1, fixed by issue 2859), and value initialization (9.5.1 [dcl.init.general] paragraph 9, also fixed by issue 2859).

2026-01-11: Split off issue 3148 for the definition of "user-declared".

Proposed resolution (approved by CWG 2026-02-06):

Change in 12.2.2.4 [over.match.ctor] paragraph 1 as follows:

When objects of class type are direct-initialized (9.5 [dcl.init]), copy-initialized from an expression of the same or a derived class type (9.5 [dcl.init]), or default-initialized (9.5 [dcl.init]), overload resolution selects the constructor. For direct-initialization or default-initialization (including default-initialization in the context of copy-list-initialization), the candidate functions are all the constructors of the class of the object being initialized. Otherwise, the candidate functions are all the non-explicit constructors (11.4.8.2 [class.conv.ctor]) of that class. The argument list is the *expression-list* or *assignment-expression* of the *initializer*; **for default-initialization, the argument list is empty**. For default-initialization in the context of copy-list-initialization, if an explicit constructor is chosen, the initialization is ill-formed.

2947. Limiting macro expansion in pp-module

Section: 15.5 [cpp.module] **Status:** ready **Submitter:** Jason Merrill **Date:** 2024-10-29

Consider:

```

#define DOT_BAR .bar
export module foo DOT_BAR;

```

The current rules appear to make this valid, referring to module `foo.bar` in phase 7. This ought to be ill-formed to align with the goal of keeping module directives parsable by simple tools.

Proposed resolution (approved by CWG 2026-03-26):

1. Change in 15.5 [cpp.module] paragraph 1 as follows:

```
pp-module:
  exportopt module pp-tokensopt ; new-line
```

The *pp-tokens*, if any, of a *pp-module* shall be of the form:

```
pp-module-name pp-module-partitionopt pp-tokensopt
```

where ~~the *pp-tokens* (if any) shall not begin with a (preprocessing token and~~ the grammar non-terminals are defined as: ...

No *identifier* in the *pp-module-name* or *pp-module-partition* shall currently be defined as an object-like macro **or followed by (as the next preprocessing token at the start of phase 4 of translation (5.2 [lex.phases])**.

2. Delete 15.5 [cpp.module] paragraph 2 as follows:

~~Any preprocessing tokens after the module preprocessing token in the module directive are processed just as in normal text. [Note 1: Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens. — end note]~~

3. Change in 15.5 [cpp.module] paragraph 3 as follows:

The module and export (if it exists) preprocessing tokens are replaced by the *module-keyword* and *export-keyword* preprocessing tokens respectively. [Note: This makes the line no longer a directive so it is not removed at the end of phase 4. — end note] **After this replacement, the preprocessing tokens that constituted the directive are a text-line and are processed as normal text. [Note: No macro expansion is possible for the *pp-module-name* and *pp-module-partition*. -- end note] After such processing, there shall be a ; or [preprocessing token following the *pp-module-name* and optional *pp-module-partition*.**

4. Add 15.5 [cpp.module] paragraph 4 with examples:

[Example:

```
// common.h
#define DOT_BAR .bar
#define MOD_ATTR [[vendor::shiny_module]]

// translation unit 1
module;
#include "common.h"

export module foo DOT_BAR; // error: expansion of DOT_BAR; does not begin with ; or [

// translation unit 2
module;
#include "common.h"

export module M MOD_ATTR ; // OK
```

-- end example]

[Example:

```
export module a
.b; // error: preprocessing token after pp-module-name is not ; or [
```

-- end example]

[Example:

```
export module M [[
attr1,
attr2 ]] ; // OK
```

-- end example]

[Example:

```
export module M
[[ attr1,
attr2 ]] ; // OK
```

-- end example]

[Example:

```
export module M; int
n; // OK
```

-- end example]

Section: 6.9.2 [[basic.fundamental](#)] **Status:** ready **Submitter:** Jiang An **Date:** 2024-11-22

(From submission [#644](#).)

While 6.9.2 [[basic.fundamental](#)] paragraph 16 specifies the size of `std::nullptr_t`, it is silent on any alignment requirements. Furthermore, unlike in C23, there is no statement about the value representation of `std::nullptr_t`.

Proposed resolution (approved by CWG 2026-01-23):

Change in 6.9.2 [[basic.fundamental](#)] paragraph 16 as follows:

The types denoted by cv `std::nullptr_t` are distinct types. A value of type `std::nullptr_t` is a null pointer constant (7.3.12 [[conv.ptr](#)]). Such values participate in the pointer and the pointer-to-member conversions (7.3.12 [[conv.ptr](#)], 7.3.13 [[conv.mem](#)]). ~~`sizeof(std::nullptr_t)` shall be equal to `sizeof(void*)`.~~ The size (7.6.2.5 [[expr.sizeof](#)]) and alignment requirement (6.8.3 [[basic.align](#)]) of the type `std::nullptr_t` are those of the type "pointer to void". [Note: The value representation can comprise no bits (7.3.2 [[conv.lval](#)]). -- end note]

2976. Transferring control out of a function

Section: 8.10 [[stmt.dcl](#)] **Status:** ready **Submitter:** Artem Koton **Date:** 2024-12-20

(From submission [#656](#).)

Subclause 8.10 [[stmt.dcl](#)] paragraph 2 specifies:

... Upon each transfer of control (including sequential execution of statements) within a function from point P to point Q, all block variables with automatic storage duration that are active at P and not at Q are destroyed in the reverse order of their construction. ...; when a function returns, Q is after its body.

The phrasing "within a function" can be misread as not applying to transfers out of a function (e.g. execution of a return statement).

Proposed resolution (approved by CWG 2025-03-25):

Change in 8.10 [[stmt.dcl](#)] paragraph 2 as follows:

A block variable with automatic storage duration (6.8.6.4 [[basic.stc.auto](#)]) is active everywhere in the scope to which it belongs after its *init-declarator*. Upon each transfer of control (including sequential execution of statements, **but excluding function calls**) within a function from point P to point Q, all block variables with automatic storage duration that are active at P and not at Q are destroyed in the reverse order of their construction. Then, all block variables with automatic storage duration that are active at Q but not at P are initialized in declaration order; unless all such variables have vacuous initialization (6.8.4 [[basic.life](#)]), the transfer of control shall not be a jump. [Footnote: ...] When a *declaration-statement* is executed, P and Q are the points immediately before and after it; when a function returns, Q is after its body.

CWG 2026-03-25

CWG believes the original issue is NAD, because the return statement that initiates the transfer of control out of a function is "within" that function.

However, CWG discovered an issue that function calls should not be handled by these rules. This is addressed in the resolution above.

2983. Non-type template parameters are not variables

Section: 6.1 [[basic.pre](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2025-01-09

Subclause 6.1 [[basic.pre](#)] paragraph 7 specifies:

A *variable* is introduced by the declaration of a reference other than a non-static data member or of an object. The variable's name, if any, denotes the reference or object.

Does this mean that non-type template parameters of class type are variables? Better not.

Possible resolution (January, 2025). [SUPERSEDED]:

Change in 6.1 [[basic.pre](#)] paragraph 7 as follows:

A non-template-parameter declaration is a declaration that is not a type-parameter or parameter-declaration of a template-parameter. A variable is introduced by the **non-template-parameter** declaration of a reference other than a non-static data member or of an object. The variable's name, if any, denotes the reference or object.

Possible resolution (approved by CWG 2026-01-09):

Change in 6.1 [[basic.pre](#)] paragraph 7 as follows:

A variable is introduced by the declaration **D** of a reference other than a non-static data member or of an object, **where D is not the parameter-declaration of a template-parameter**. The variable's name, if any, denotes the reference or object.

2992. Labels do not have names

Section: 6.1 [\[basic.pre\]](#) **Status:** ready **Submitter:** Vlad Serebrennikov **Date:** 2025-02-20

(From submission [#673](#).)

Subclause 6.1 [\[basic.pre\]](#) paragraph 3 specifies:

A name is an *identifier* (5.11 [\[lex.name\]](#)), *conversion-function-id* (11.4.8.3 [\[class.conv.fct\]](#)), *operator-function-id* (12.4 [\[over.oper\]](#)), or *literal-operator-id* (12.6 [\[over.literal\]](#)).

That seems to incorrectly include identifier labels per the grammar in 8.2 [\[stmt.label\]](#) paragraph 1 as well as attributes per the grammar in 9.13.1 [\[decl.attr.grammar\]](#) paragraph 1.

Proposed resolution (approved by CWG 2026-03-06):

Change in 6.1 [\[basic.pre\]](#) paragraph 3 as follows and add bullets:

A name is

- an *identifier* **token** (5.10 [\[lex.token\]](#), 5.11 [\[lex.name\]](#)) **other than**
 - the *identifier* of a label (8.2 [\[stmt.label\]](#)) or *literal-operator-id* (12.6 [\[over.literal\]](#)),
 - the *identifier* following a goto in a jump-statement (8.8.1 [\[stmt.jump.general\]](#)),
 - any *identifier* in a module-name (10.1 [\[module.unit\]](#)) or attribute-token (9.13.1 [\[decl.attr.grammar\]](#)), or
- a *conversion-function-id* (11.4.8.3 [\[class.conv.fct\]](#)),
- an *operator-function-id* (12.4 [\[over.oper\]](#)), or
- a *literal-operator-id* (12.6 [\[over.literal\]](#)).

3010. constexpr placement-new should require transparent replaceability

Section: 7.7 [\[expr.const\]](#) **Status:** ready **Submitter:** Richard Smith **Date:** 2025-03-16

Consider:

```
struct A { int n; };
struct B : A { int m; };

constexpr int f() {
    B b = {{0}, 0};
    A *p = &b;
    new (p) A{1};    // does not transparently replace *p
    return p->n;     // UB, p refers to out-of-lifetime object
}
constexpr int k = f();
```

It seems unreasonable to expect implementations to track situations where a placement-new evaluations that don't transparently replace the original object.

The status quo wording in 7.7 [\[expr.const\]](#) bullet 10.18.2 involving "similar" was added by issue 2922.

Proposed resolution (approved by CWG 2026-02-20):

Change in 7.7 [\[expr.const\]](#) bullet 10.18.2 as follows:

An expression E is a core constant expression unless the evaluation of E, following the rules of the abstract machine (6.10.1 [\[intro.execution\]](#)), would evaluate one of the following:

- ...
- a *new-expression* (7.6.2.8 [\[expr.new\]](#)), unless either
 - the selected allocation function is a replaceable global allocation function (17.6.3.2 [\[new.delete.single\]](#), 17.6.3.3 [\[new.delete.array\]](#)) and the allocated storage is deallocated within the evaluation of E, or
 - the selected allocation function is a non-allocating form (17.6.3.4 [\[new.delete.placement\]](#)) with an allocated type T, where
 - the placement argument to the *new-expression* points to an object **whose type is similar to T (7.3.6 [\[constexpr\]](#)) that is transparently replaceable (6.8.4 [\[basic.life\]](#)) by the object created by the *new-expression*** or, if T is an array type, to the first element of **such** an object **of a type similar to T**, and
 - the placement argument points to storage whose duration began within the evaluation of E;
- ...

3029. Confusing note about ordinary character types for aligned memory areas

Section: 6.8.3 [\[basic.align\]](#) **Status:** ready **Submitter:** Xavier Bonaventura **Date:** 2025-03-20

(From submission [#687](#).)

Subclause 6.8.3 [\[basic.align\]](#) paragraph 6 specifies:

The alignment requirement of a complete type can be queried using an alignof expression (7.6.2.6 [expr.alignof]). Furthermore, the narrow character types (6.9.2 [basic.fundamental]) shall have the weakest alignment requirement. [Note 2: This enables the ordinary character types to be used as the underlying type for an aligned memory area (9.13.2 [dcl.align]). —end note]

The note has multiple problems: First, the note talks about "ordinary character types" (including char8_t; see 6.9.2 [basic.fundamental] paragraph 7), whereas the immediately preceding sentence is limited to "narrow character types". Second, due to memory model concerns, signed char is not suitable for aligned memory areas providing storage for other objects.

Proposed resolution (no objections during CWG reflector review starting 2025-05-11) [SUPERSEDED]:

Change in 6.8.3 [basic.align] paragraph 6 as follows:

The alignment requirement of a complete type can be queried using an alignof expression (7.6.2.6 [expr.alignof]). Furthermore, the narrow character types (6.9.2 [basic.fundamental]) shall have the weakest alignment requirement. [Note 2: This enables the ordinary character types to be used as the underlying type for an aligned memory area (9.13.2 [dcl.align]). —end note]

Additional notes (June, 2025)

The "underlying type" is for enumerations and derived integer types and thus is the wrong phrase to use here.

Proposed resolution (approved by CWG 2026-01-09):

Change in 6.8.3 [basic.align] paragraph 6 as follows:

The alignment requirement of a complete type can be queried using an alignof expression (7.6.2.6 [expr.alignof]). Furthermore, the narrow character types (6.9.2 [basic.fundamental]) shall have the weakest alignment requirement. [Note 2: This enables the ordinary character types to be used as the underlying type for an aligned memory area (9.13.2 [dcl.align]). —end note]

3035. Lambda expressions in anonymous unions

Section: 11.5.2 [class.union.anon] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-05-12

(From submission #706.)

Consider:

```
static union {  
    int x = [] { return 42; }();  
};
```

According to 11.5.2 [class.union.anon] paragraph 1, this is ill-formed, because the closure type is declared as a nested type of the anonymous union (7.5.6.2 [expr.prim.lambda.closure] paragraph 3, but some implementations do not yield a diagnostic.

On the other hand, the code shown in the example is reasonable and ought to be allowed.

Proposed resolution (approved by CWG 2026-02-06):

Change in 11.5.2 [class.union.anon] paragraph 1 as follows:

... Nested types: (including closure types (7.5.6.2 [expr.prim.lambda.closure]) and anonymous unions;) and functions shall not be declared within an anonymous union....

3058. "Program point" is not defined

Section: 6.5.1 [basic.lookup.general] **Status:** ready **Submitter:** Alisdair Meredith **Date:** 2025-08-13 N5028 comment US 14-029

(From submission #749.)

There is no definition for the term "program point".

Suggested resolution [SUPERSEDED]:

Change in 6.5.1 [basic.lookup.general] paragraph 2 as follows:

There is a *program point* before the first token of the translation unit, between every pair of adjacent tokens, and after the last token of the translation unit. A program point *P* is said to follow any declaration in the same translation unit whose locus (6.4.2 [basic.scope.pdecl]) is before *P*. [Note 1: The declaration might appear in a scope that does not contain *P*. —end note] ...

Suggested resolution [SUPERSEDED]:

There is a *program point* before the first token of the translation unit, an arbitrary number between every pair of adjacent tokens, and an arbitrary number after the last token of the translation unit. A program point *P* is said to follow any declaration in the same translation unit whose locus (6.4.2 [basic.scope.pdecl]) is before *P*. [Note 1: The declaration might appear in a scope that does not contain *P*. —end note] ...

Proposed resolution (approved by CWG 2025-12-05):

There is a *program point* before the first token of the translation unit, at least one between every pair of adjacent tokens, and at least one after the last token of the translation unit. A program point *P* is said to follow any declaration in the same translation unit whose locus (6.4.2 [basic.scope.pdecl]) is before *P*. [Note 1: The declaration might appear in a scope that does not contain *P*. — end note] ...

3088. Clarify macro treatment of identifiers with special meaning

Section: 15.7.1 [cpp.replace.general] **Status:** ready **Submitter:** US **Date:** 2025-10-01 [N5028 comment US 57-105](#)

Subclause C.1.6 [diff.cpp23.library] paragraph 2 specifies:

Affected subclause: 16.4.6.3 [res.on.macro.definitions]

Change: Additional restrictions on macro names.

Rationale: Avoid hard to diagnose or non-portable constructs.

Effect on original feature: Names of special identifiers may not be used as macro names. Valid C++ 2023 code that defines replaceable_if_eligible or trivially_relocatable_if_eligible as macros is invalid in this revision of C++.

The restriction on defining keywords and identifiers with special meaning as macros was moved to 15.7.1 [cpp.replace.general] paragraph 9.

Proposed resolution (reviewed by CWG 2025-11-07) [SUPERSEDED]:

1. Insert before 5.11 [lex.name] paragraph 3 as follows:

[Note: Identifiers with special meaning cannot be used as macro names (15.7.1 [cpp.replace.general]). -- end note]

In addition, some identifiers appearing as a token or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required. ...

2. Insert before 5.12 [lex.key] paragraph 2 as follows:

[Note: Keywords cannot be used as macro names (15.7.1 [cpp.replace.general]). -- end note]

Furthermore, the alternative representations shown in Table 6 for certain operators and punctuators (5.9 [lex.digraph]) are reserved and shall not be used otherwise.

3. Insert before 9.13.1 [dcl.attr.grammar] paragraph 5 as follows:

[Note: Unless otherwise specified, an *attribute-token* specified in this document cannot be used as a macro name (15.7.1 [cpp.replace.general]). -- end note]

An annotation followed by an ellipsis is a pack expansion (13.7.4 [temp.variadic]).

4. Change in 15.7.1 [cpp.replace.general] paragraph 9 as follows:

A translation unit shall not #define or #undef names lexically identical to keywords (5.12 [lex.key]), to the identifiers listed in Table 4, or to the *attribute-tokens* described in 9.13 [dcl.attr], except that the names likely and unlikely may be defined as function-like macros.

5. Move C.1.6 [diff.cpp23.library] paragraph 2 to a new clause C.1.5+ "Clause 15 preprocessing directives" and change as follows:

Affected subclause: ~~16.4.6.3 [res.on.macro.definitions]~~ 15.7.1 [cpp.replace.general]

Change: Additional restrictions on macro names.

Rationale: Avoid hard to diagnose or non-portable constructs.

Effect on original feature: Names of ~~special identifiers~~ **identifiers with special meaning** (5.11 [lex.name]) may not be used as macro names. Valid C++ 2023 code that defines replaceable_if_eligible or trivially_relocatable_if_eligible as macros is invalid in this revision of C++.

Proposed resolution (approved by CWG 2026-01-23):

1. Insert before 5.11 [lex.name] paragraph 3 as follows:

[Note: Identifiers with special meaning cannot be used as macro names (15.7.1 [cpp.replace.general]). -- end note]

In addition, some identifiers appearing as a token or *preprocessing-token* are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required. ...

2. Insert before 5.12 [lex.key] paragraph 2 as follows:

[Note: Keywords cannot be used as macro names (15.7.1 [cpp.replace.general]). -- end note]

Furthermore, the alternative representations shown in Table 6 for certain operators and punctuators (5.9 [lex.digraph]) are reserved and shall not be used otherwise.

3. Insert before 9.13.1 [dcl.attr.grammar] paragraph 5 as follows:

[Note: Unless otherwise specified, an *attribute-token* specified in this document cannot be used as a macro name (15.7.1 [cpp.replace.general]). -- end note]

An annotation followed by an ellipsis is a pack expansion (13.7.4 [temp.variadic]).

4. Change in 15.7.1 [cpp.replace.general] paragraph 9 as follows:

A translation unit shall not `#define` or `#undef` macro names lexically identical to keywords (5.12 [lex.key]), to the identifiers listed in Table 4, or to the *attribute-tokens* described in 9.13 [dcl.attr], except that the macro names likely and unlikely may be defined as function-like macros and may be undefined.

5. Add a new clause C.1.5+ "Clause 15 preprocessing directives" with the following content:

Affected subclause: 15.7.1 [cpp.replace.general]

Change: Additional restrictions on macro names.

Rationale: Avoid hard to diagnose or non-portable constructs.

Effect on original feature: Keywords, names of identifiers with special meaning (5.11 [lex.name]), and (unless otherwise specified) *attribute-tokens* specified in 9.13 [dcl.attr] may not be used as macro names. For example, valid C++ 2023 code that defines `post` or `pre` as macros is invalid in this revision of C++.

3119. *for-range-declaration* of an *expansion-statement* as a templated entity

Section: 13.1 [temp.pre] **Status:** ready **Submitter:** AT **Date:** 2025-10-01 [N5028 comment AT 2-089](#)

Currently, a structured binding pack can only be declared in the *for-range-declaration* of an *expansion-statement* if there is an enclosing template. Make entities declared therein templated entities.

Consider:

```
struct B
{
    int i;
    long l;
};

struct A
{
    B b;
};

void f(int i, long l);

int main()
{
    template for (auto [ ... e] : A()) {
        f(e...);    // OK
    }
}
```

Proposed resolution (approved by CWG 2025-11-08):

Change in 13.1 [temp.pre] bullet 8.2 as follows:

An entity is templated if it is

- a template,
- an entity defined (6.2 [basic.def]) or created (6.8.7 [class.temporary]) within the *for-range-declaration* or *compound-statement* of an *expansion-statement* (8.7 [stmt.expand]),
- ...

3122. Inadequate value-dependence for *reflect-expressions*

Section: 13.8.3.4 [temp.dep.constexpr] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-11-15

(From submission [#808](#).)

The follow example ought to be value-dependent, but is currently not specified to be so:

```
template <typename T>
constexpr void f() {
    T v;
    std::meta::info R = std::meta::type_of(^v);    // not currently value-dependent
}
```

Proposed resolution (approved by CWG 2025-11-21):

Change in 13.8.3.4 [temp.dep.constexpr] paragraph 7 as follows:

A *reflect-expression* is value-dependent if

- it is of the form `^^reflection-name` and either lookup for the *reflection-name* finds a declaration that inhabits a scope corresponding to a templated entity or the *reflection-name* is
 - ~~is~~ a dependent qualified name,
 - ~~is~~ a dependent *namespace-name*, or
 - ~~is~~ the name of a template parameter, or

- names a dependent member of the current instantiation (13.8.3.2 [temp.dep.type]);
- it is of the form `^type-id` and the *type-id* denotes a dependent type, or
- it is of the form `^id-expression` and the *id-expression* is value-dependent.

(Drafting note: Reflection on block-scope externs is made ill-formed by issue 3065.)

3123. Global lookup for `begin` and `end` for expansion statements

Section: 8.7 [stmt.expand] **Status:** ready **Submitter:** Ambrose T. **Date:** 2025-11-11

(From submission #803.)

Subclause 8.7 [stmt.expand] paragraph 3 specifies:

For an expression *E*, let the expressions *begin-expr* and *end-expr* be determined as specified in 8.6.5 [stmt.ranged]. An expression is *expansion-iterable* if it does not have array type and either

- begin-expr* and *end-expr* are of the form *E*.begin() and *E*.end(), or
- argument-dependent lookups for begin(*E*) and for end(*E*) each find at least one function or function template.

For the non-member case, this means any `begin` and `end` function will make an expression *expansion-iterable*. For example, a `std::tuple` is specified to be *expansion-iterable*, but it ought actually to be treated by destructuring.

Other appearances of "find at least one declaration" in the standard apply to member lookup only, e.g. in 7.6.2.4 [expr.await] bullet 3.2 or 9.7 [decl.struct.bind] paragraph 7; the intent there is to "lock into" the member interpretation, giving a diagnostic if the `begin` or `end` member is not quite matching, e.g. due to missing `const`.

Proposed resolution (approved by CWG 2026-01-23):

Change in 8.7 [stmt.expand] paragraph 3 as follows:

For an expression *E*, let the expressions *begin-expr* and *end-expr* be determined as specified in 8.6.5 [stmt.ranged]. An expression is *expansion-iterable* if it does not have array type and either

- begin-expr* and *end-expr* are of the form *E*.begin() and *E*.end(), or
- argument-dependent lookups for begin(*E*) and for end(*E*) each find at least one **function or function template viable candidate (12.2.3 [over.match.viable])**.

Additional notes (CWG 2026-01-23)

There is agreement that `std::tuple` should not be considered *expansion-iterable*, and the proposed resolution achieves that goal. However, the proposed resolution can also cause additional template instantiations, which might be needed to check convertibility of arguments to parameters, even if the original expression is eventually deemed not *expansion-iterable*. A template instantiation might fail in a non-immediate context. A template instantiation can have non-local effects later observable via reflection.

The consideration whether the approach as presented, given its drawbacks, is preferable over other approaches with different drawbacks is an EWG design concern. CWG requests EWG to weigh in via [paper issue #2615](#).

EWG 2026-03-23

EWG has approved P4026R0, choosing the "viable candidate" approach (see proposed resolution above).

3124. Disallow annotations on block-scope externs and non-unique friend declarations

Section: 9.13.12 [decl.attr.annotation] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-11-15

(From submission #809.)

Be conservative and disallow annotations on block-scope externs.

Suggested resolution (option 1, 2025-11): [SUPERSEDED]

1. Change in 9.13.1 [decl.attr.grammar] paragraph 6 as follows:

Each *attribute-specifier-seq* is said to appertain to some entity or statement, identified by the syntactic context where it appears (Clause Clause 8 [stmt], Clause Clause 9 [decl], 9.3 [decl.decl]). If an *attribute-specifier-seq* that appertains to some entity or statement contains an attribute or *alignment-specifier* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.8.4 [class.friend]), that declaration shall be a definition. **Furthermore, if an annotation applies to a friend declaration *D*, or to a function parameter declaration in the function-declarator (if any) of *D*, then for any other declaration *D'* of the entity declared by *D*, neither *D* nor *D'* shall be reachable from the other.**

2. Change in 9.13.12 [decl.attr.annotation] paragraph 1 as follows:

An annotation may be applied to **any** a declaration *D* of a type, type alias, variable, function, namespace, enumerator, *base-specifier*, or non-static data member. **If an annotation applies to a declaration *D*, or to a function parameter declaration in the function-declarator (if any) of *D*, then the host scope of *D* (6.4.1 [basic.scope.scope]) shall be the target scope of *D*.**

1. Change in 9.13.12 [dcl.attr.annotation] paragraph 1 as follows:

The *surrounding declaration* of a declaration *D* is

- *D'* if *D* is a function parameter declaration in a *function-declarator* of a function declaration *D'* and
- *D* otherwise.

An annotation may be applied to **any** a declaration ***D*** of a type, type alias, variable, function, namespace, enumerator, *base-specifier*, or non-static data member, **unless**

- the host scope of the surrounding declaration of *D* differs from its target scope or
- the surrounding declaration *D'* of *D* is a friend declaration and another declaration for the entity declared by *D'* is reachable from *D'* or vice versa.

CWG 2026-02-06

CWG preferred the wording of option 2. Instead of the condition involving mutual reachability, CWG preferred the more traditional "is a definition" restriction.

Proposed resolution (approved by CWG 2026-03-26)

Change in 9.13.12 [dcl.attr.annotation] paragraph 1 as follows:

An annotation may be applied to **any** a declaration ***D*** of a type, type alias, variable, function, namespace, enumerator, *base-specifier*, or non-static data member, **unless**

- the host scope of *X* differs from its target scope or
- *X* is a non-defining friend declaration,

where *X* is

- *D'* if *D* is a function parameter declaration in a *function-declarator* of a function declaration *D'* and
- *D* otherwise.

3125. Token convertibility requirement in #if

Section: 15.2 [cpp.cond] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-11-18

(From submission #813.)

Consider:

```
#define F(X) X
#if __has_include(F(<#.h>))
#endif
```

This is ill-formed per 15.2 [cpp.cond] paragraph 8, even though the # preprocessing token will eventually be used to form a *header-name* and thus will not be converted into a *token* per 15.2 [cpp.cond] paragraph 11. The latter is governed by 5.5 [lex.pptoken] paragraph 2.

Proposed resolution (approved by CWG 2025-12-05):

Change in 15.2 [cpp.cond] paragraph 8 as follows:

~~Each preprocessing token that remains (in the list of preprocessing tokens that will become the controlling expression) after all macro replacements have occurred shall be in the lexical form of a token (5.10 [lex.token]).~~

3126. A module import needs a header-name as a token

Section: 5.10 [lex.token] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-11-18

(From submission #812.)

Consider:

```
import "myheader.h";
```

The (phase 7) grammar *module-import-declaration* (10.3 [module.import]) expects a *header-name*, but that is not a valid *token* per 5.10 [lex.token].

Proposed resolution (approved by CWG 2025-12-05):

1. Change in 5.5 [lex.pptoken] paragraph 2 as follows:

Each preprocessing token that is converted to a token (5.10 [lex.token]) shall have the lexical form of a keyword, an identifier, a literal, **a header name**, or an operator or punctuation.

2. Change in 5.10 [lex.token] as follows:

```
token:
    identifier
```

keyword
literal
header-name
operator-or-punctuator

There are five six kinds of tokens: identifiers, keywords, literals [Footnote: ...], header names, operators, and other separators. ...

3127. Evaluation context is missing synthesized points

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Dan Katz **Date:** 2025-11-15

(From submission #810.)

Consider:

```
template <typename T>
struct TCls {
    T mem;
    static_assert(sizeof(T) > 0); // <-- problem
};

struct S;
constexpr {
    constexpr auto ctx = std::meta::access_context::current();

    if (nonstatic_data_members_of(substitute(TCls, {define_aggregate(S, {})}), ctx).size() > 1)
        throw "contrived, but whatever";
}
```

Nothing in the definition of "evaluation context" takes the synthesized point associated with S from the instantiation context of TCls<S> into the evaluation context of the static assertion.

Possible resolution [SUPERSEDED]:

Change in 7.7 [expr.const] paragraph 32 as follows:

... During the evaluation V of an expression E as a core constant expression, the evaluation context of an evaluation X (6.10.1 [intro.execution]) consists of the following points:

- The program point EVAL-PT(L) and any synthesized point in the instantiation context thereof, where L is the point at which E appears, and where EVAL-PT(P), for a point P, is a point R determined as follows:
 - ...
- Each synthesized point corresponding to an injected declaration produced by any evaluation sequenced before X (6.10.1 [intro.execution]).

CWG 2026-03-06

This issue is resolved by the resolution of issue 3162.

3128. Potentially-throwing unevaluated operands

Section: 14.5 [except.spec] **Status:** ready **Submitter:** Richard Smith **Date:** 2018-05-06

Consider:

```
void f();
bool b = noexcept(noexcept(f())); // potentially throwing
// ought to be "true"
```

The specification does not properly handle subexpressions of the noexcept operator that are not potentially evaluated (and thus cannot contribute to any exception throwing).

Proposed resolution (approved by CWG 2025-12-05):

Change in 14.5 [except.spec] paragraph 6 as follows:

An expression E is *potentially-throwing* if

- ...
- any of the immediate subexpressions (6.10.1 [intro.execution]) of E that is not an unevaluated operand is potentially-throwing.

3129. Clarify which floating-point-literals are valid

Section: 5.13.4 [lex.fcon] **Status:** ready **Submitter:** Jan Schultke **Date:** 2025-11-10

(From submission #801.)

```
float x = 1e1000000000000000000000000000000000000000000000000.f;
```

Proposed resolution (approved by CWG 2026-03-27):

If the scaled value is not in the range of representable values for its type, the program is ill-formed. Otherwise, the value of a *floating-point-literal* is the scaled value if representable, else the larger or smaller representable value nearest the scaled value, chosen in an implementation-defined manner.

The following example assumes that `std::float32_t` is supported (6.9.3 [basic.extended.fp]).

Additional notes (March, 2026)

The above resolution was approved by SG6 and EWG.

Section: 7.5.5.1 [[expr.prim.id.general](#)] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-11-11

Subclause 7.5.5.1 [\[expr.prim.id.general\]](#) paragraph 3 does not properly handle an *id-expression* that denotes a function member of a namespace-scope anonymous union. With reflection, those can be named.

Change in 7.5.5.1 [[expr.prim.id.general](#)] bullet 3.1 as follows:

If an *id-expression* E denotes a **variant** member M of an anonymous union (11.5.2 [class.union.anon]) U:

- ...

CWG 2025-11-21

Additional discussion has shown many issues arising from the ability to designate anonymous unions via reflection, inspect its implicitly-declared special member functions, and create complete objects of anonymous union type. Not declaring those special member functions in the first place resolves most of these concerns; users cannot declare member functions of anonymous unions anyway. CWG is interested in adding a type trait (both at the type and reflection levels) that identifies anonymous unions.

Proposed resolution (approved by CWG 2026-02-06):

1. Change in 7.5.5.1 [[expr.prim.id.general](#)] bullet 3.1 as follows:

If an *id-expression* E denotes a **variant** member M of an anonymous union (11.5.2 [class.union.anon]) U:

- ...

[*Note 3*: Under this interpretation, E no longer denotes a non-static data member. —*end note*] [*Example 3*: N::x is interpreted as N::u.x, where u names the anonymous union variable. —*end example*]

2. Change in 11.4.5.2 [[class.default.ctor](#)] paragraph 1 as follows:

... If there is no user-declared constructor or constructor template for class X **and X is not an anonymous union**, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.6 [dcl.fct.def]). An implicitly-declared default constructor is an inline public member of its class.

3. Change in 11.4.5.3 [class.copy.ctor] paragraph 6 as follows:

If the class definition does not explicitly declare a copy constructor **and the class is not an anonymous union**, a non-explicit one is declared implicitly. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defaulted (9.6 [dcl.fct.def]). The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor (D.6 [depr.impldec]).

4. Change in 11.4.5.3 [class.copy.ctor] paragraph 8 as follows:

- **X is not an anonymous union.**

- X does not have a user-declared copy constructor,
- X does not have a user-declared copy assignment operator,
- X does not have a user-declared move assignment operator, and
- X does not have a user-declared destructor.

5. Change in 11.4.6 [\[class.copy.assign\]](#) paragraph 2 as follows:

If the class definition does not explicitly declare a copy assignment operator **and the class is not an anonymous union**, one is declared implicitly. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defaulted (9.6 [\[dcl.fct.def\]](#)). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor (D.6 [\[depr.impldec\]](#)). ...

6. Change in 11.4.6 [\[class.copy.assign\]](#) paragraph 4 as follows:

If the definition of a class X does not explicitly declare a move assignment operator, one will be implicitly declared as defaulted if and only if

- X is not an anonymous union,
- X does not have a user-declared copy constructor,
- X does not have a user-declared move constructor,
- X does not have a user-declared copy assignment operator, and
- X does not have a user-declared destructor.

7. Change in 11.4.7 [\[class.dtor\]](#) paragraph 2 as follows:

If a class has no user-declared prospective destructor **and the class is not an anonymous union**, a prospective destructor is implicitly declared as defaulted (9.6 [\[dcl.fct.def\]](#)). An implicitly-declared prospective destructor is an inline public member of its class.

8. Change in 11.4.7 [\[class.dtor\]](#) paragraph 4 as follows:

At the end of the definition of a class **other than an anonymous union**, overload resolution is performed among the prospective destructors declared in that class with an empty argument list to select the destructor for the class, also known as the selected destructor. The program is ill-formed if overload resolution fails. Destructor selection does not constitute a reference to, or odr-use (6.3 [\[basic.def.odr\]](#)) of, the selected destructor, and in particular, the selected destructor may be deleted (9.6.3 [\[dcl.fct.def.delete\]](#)).

9. Change in 11.5.2 [\[class.union.anon\]](#) paragraph 1 as follows:

A union of the form

```
union { member-specification } ;
```

is called an anonymous union; it defines an unnamed type and an unnamed object of that type called an anonymous union member if it is a non-static data member or an anonymous union variable otherwise. **Each object of such an unnamed type shall be such an unnamed object.** Each *member-declaration* in the *member-specification* of an anonymous union shall either define one or more public non-static data members or be a *static_assert-declaration*. Nested types, anonymous unions, and functions shall not be declared within an anonymous union. The names of the members of an anonymous union are bound in the scope inhabited by the union declaration.

3131. Value categories and types for the range in iterable expansion statements

Section: 8.7 [\[stmt.expand\]](#) **Status:** ready **Submitter:** Jakub Jelinek **Date:** 2025-11-13

(From submission [#805](#).)

Consider:

```
#include <span>

constexpr int arr[3] = { 1, 2, 3 };
constexpr std::span<const int> foo() {
    return std::span<const int>(arr);
}

int main() {
    int r = 0;
    template for (constexpr auto m : foo())
        r += m;
}
```

The expansion of this iterable expansion statement contains:

```
constexpr auto &&range = foo();
```

The deduction yields `std::span<const int>&&` for the type of `range`, and the non-const lifetime-extended temporary causes later constant evaluation in the expansion to fail.

Proposed resolution (approved by CWG 2026-03-26):

Change in 8.7 [\[stmt.expand\]](#) bullet 5.2 as follows:

- ...
- Otherwise, if S is an iterating expansion statement, S is equivalent to:


```
{
    init-statement
```

```

constexpr auto&& decltype(auto) range = (expansion-initializer) ;
constexpr auto begin = begin-expr; // see 8.6.5 [stmt.ranged]
constexpr auto end = end-expr; // see 8.6.5 [stmt.ranged]
S0
.
.
.
SN-1
}

```

where N is ...

3132. Unclear disambiguation rule for *condition*

Section: 8.1 [stmt.pre] **Status:** ready **Submitter:** CWG **Date:** 2025-11-21

The disambiguation rule in 8.1 [stmt.pre] paragraph 7 is unclear, because "declaration" is not a (uniquely defined) grammatical category.

Proposed resolution (approved by CWG 2025-12-05):

1. Change in 8.1 [stmt.pre] paragraph 1 as follows:

```

condition :
    expression
    condition-declaration
condition-declaration :
    attribute-specifier-seqopt decl-specifier-seq declarator brace-or-equal-initializer
    structured-binding-declaration initializer

```

2. Change in 8.1 [stmt.pre] paragraph 7 as follows:

If a *condition* can be syntactically resolved as either an *expression* or a *condition-declaration*, it is interpreted as the latter.

3133. Cv-qualified types in built-in operator candidates

Section: 12.5 [over.built] **Status:** ready **Submitter:** Jan Schultke **Date:** 2025-11-10

(From submission #800.)

Subclause 12.5 [over.built] is inconsistent in its inclusion or exclusion of cv-qualified types and permits them even where the parameter-type-list is not affected.

Proposed resolution (approved by CWG 2026-02-06):

1. Change in 12.5 [over.built] paragraph 10 as follows:

For every pair of types L and R, where each of L and R is a *cv-unqualified* floating-point or promoted integral type, there exist candidate operator functions of the form ...

2. Change in 12.5 [over.built] paragraph 11 as follows:

For every *cv-unqualified* integral type T there exists a candidate operator function of the form ...

3. Change in 12.5 [over.built] paragraph 12 as follows:

For every pair of *cv-unqualified* floating-point types L and R, there exists a candidate operator function of the form ...

4. Change in 12.5 [over.built] paragraph 18 as follows:

For every triple (L, vq, R), where L is a *cv-unqualified* arithmetic type and R is a *cv-unqualified* floating-point or promoted integral type, there exist candidate operator functions of the form ...

5. Change in 12.5 [over.built] paragraph 22 as follows:

For every triple (L, vq, R), where L is a *cv-unqualified* integral type and R is a promoted integral type, there exist candidate operator functions of the form ...

6. Change in 12.5 [over.built] paragraph 24 as follows:

For every pair of types L and R, where each of L and R is a *cv-unqualified* floating-point or promoted integral type, there exist candidate operator functions of the form ...

3135. constexpr structured bindings with prvalues from tuples

Section: 9.7 [\[dcl.struct.bind\]](#) **Status:** ready **Submitter:** Barry Revzin **Date:** 2025-11-14

(From submission [#807](#).)

In the tuple case, `e.get<i>()` or `get<i>(e)` is used, but the case that those function calls return a prvalue is not properly handled for `constexpr`. For example, P1789R3 (Library Support for Expansion Statements) added `constexpr T get(integer_sequence<...>)`.

Specific example:

```
constexpr auto [...Is] = std::make_index_sequence<2>();
```

decomposes into:

```
constexpr auto e = std::integer_sequence<size_t, 0, 1>();
constexpr size_t&& Is#0 = 0; // temporary is mutable
constexpr size_t&& Is#1 = 1;
```

Structured bindings for prvalues should use a non-reference type.

Proposed resolution (approved by CWG 2026-03-27):

Change in 9.7 [\[dcl.struct.bind\]](#) paragraph 7 as follows:

... Given the type `Ti` designated by `std::tuple_element<i, E>::type` and the type `Ui` designated by either `Ti` or `Ti&&` where `Ui` is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise, defined as `Ti` if the initializer is a prvalue, as "lvalue reference to `Ti`" if the initializer is an lvalue, or as "rvalue reference to `Ti`" otherwise, variables are introduced with unique names `ri` as follows:

```
S Ui ri = initializer ;
```

3136. Constant expressions of type void

Section: 7.7 [\[expr.const\]](#) **Status:** ready **Submitter:** Lénárd Szolnoki **Date:** 2025-11-23

(From submission [#821](#).)

Subclause 7.7 [\[expr.const\]](#) bullet 22.2 does not handle expressions of type void or of scalar types.

Proposed resolution (approved by CWG 2026-02-06):

Change in 7.7 [\[expr.const\]](#) bullet 22.2 as follows:

A constant expression is either

- a glvalue core constant expression `E` for which ...
- a prvalue core constant expression whose result object (7.2.1 [\[basic.lval\]](#)) (if any) satisfies the following constraints: ...

Drafting note: A prvalue of object type always has a result object; see 7.2.1 [\[basic.lval\]](#) paragraph 5.

3140. Allowing expansion over non-constant `std::array`

Section: 8.7 [\[stmt.expand\]](#) **Status:** ready **Submitter:** Barry Revzin **Date:** 2025-12-04

(From submission [#805](#).)

(Split off from issue 3131.)

In order to allow expansion over non-constant `std::array` (which does have a compile-time size), the `constexpr` in the range declaration in 8.7 [\[stmt.expand\]](#) bullet 5.2 should be optional, similar to the destructuring case.

Proposed resolution (approved by CWG 2026-03-27):

1. Change in 6.8.7 [\[class.temporary\]](#) paragraph 9 (as modified by issue 3043) as follows:

The sixth context is when a temporary object is created in the *expansion-initializer* of an iterating or a destructuring expansion statement. If such a temporary object would otherwise be destroyed at the end of that *expansion-initializer*, the object persists for the lifetime of the reference initialized by the *expansion-initializer*, if any.

2. Change in 8.7 [\[stmt.expand\]](#) bullet 5.2 as follows:

```
◦ ...
◦ Otherwise, if S is an iterating expansion statement, S is equivalent to:

{
    init-statement
    constexpropt auto&& range = expansion-initializer ;
    constexpropt auto begin = begin-expr; // see 8.6.5 \[stmt.ranged\]
    constexpr auto end = end-expr; // see 8.6.5 \[stmt.ranged\]
```

```

S0
.
.
.
SN-1
}

```

where N is the result of evaluating the expression

```

[&] consteval {
    std::ptrdiff_t result = 0;
    for (auto i = begin; i != end; ++i, ++result);
    auto b = begin-expr;
    auto e = end-expr;
    for (; b != e; ++b) ++result;
    return result; // distance from begin to end
}()

```

and Si is

```

{
    constexpr_opt auto iter = begin + i;
    for-range-declaration = *iter;
    compound-statement
}

```

The variables range, begin, end, and iter are defined for exposition only. The keyword constexpr is present in the declarations of range, begin, and iter if and only if constexpr is one of the decl-specifiers of the decl-specifier-seq of the for-range-declaration. [Note 1: The instantiation is ill-formed if range is not a constant expression (7.7 [expr.const]). — end note]

o ...

3141. Unique objects from define_static_array

Section: 6.8.2 [intro.object] **Status:** ready **Submitter:** A. Jiang **Date:** 2025-11-29

Subclause 6.8.2 [intro.object] bullet 9.3 specifies that any objects resulting from define_static_array are potentially overlapping (i.e. may have a non-unique address), include std::array<T, 0>.

Subclause 13.2 [temp.param] paragraph 13 (after issue 3111) specifies that template parameter objects are distinct.

Subclause 21.4.3 [meta.define.static] specifies that reflect_static_array results in a template parameter object.

These requirements appear to be contradictory. However, it was observed that "distinct object" is referring to the abstract machine notion of an object; objects can be distinct (resulting in different std::meta::info values when reflected upon) even if they have the same address.

It was further observed that only reflection facilities produce array values as template parameter objects; template arguments do not. It is design-intended that such array values can share storage with string literals and backing arrays of initializer lists.

Proposed resolution (approved by CWG 2026-01-09):

1. Change in 6.8.2 [intro.object] bullet 9.3 as follows:

An object is a *potentially non-unique object* if it is

- o a string literal object (5.13.5 [lex.string]),
- o the backing array of an initializer list (9.5.4 [dcl.init.ref]), or
- o the object introduced by a call to std::meta::reflect_constant_array or std::meta::reflect_constant_string a template parameter object of array type (21.4.3 [meta.define.static]), or
- o a subobject thereof.

2. Change in 13.2 [temp.param] paragraph 13 as follows:

[Note: There can be template parameter objects of array type (21.4.3 [meta.define.static]), but such an object is never denoted by an *id-expression* that names a constant template parameter. Such an object can have an address that is not unique among all other in-lifetime objects (6.8.2 [intro.object]). -- end note]

3142. Possible expansions of __LINE__ changing over time

Section: 15.12 [cpp.predefined] **Status:** ready **Submitter:** Jiang An **Date:** 2025-12-05

(From submission #826.)

__LINE__ is specified in 15.12 [cpp.predefined] paragraph 1 as:

The presumed line number (within the current source file) of the current source line (an integer literal).

The spelling of a literal is observable through stringizing (15.7.3 [cpp.stringize]). Furthermore, the set of possible spellings for a literal of a given integer value has expanded over time; for example digit separators and binary integer literals are permitted since C++14.

A program that relies on a particular spelling of an integer literal expanded from `__LINE__` might thus break, e.g. when crafting unique identifiers using token concatenation (15.7.4 [cpp.concat]).

We could either add an Annex C entry for this breakage in C++14, or specify that `__LINE__` always yields a decimal integer literal with no digit separators.

See also WG14 [issue 1016](#).

Proposed resolution (approved by CWG 2026-03-26):

Change in 15.12 [cpp.predefined] as follows:

`__LINE__`
~~An~~ **0 or a decimal** integer literal (5.13.2 [lex.icon]), with no digit separators and no *integer-suffix*, representing the presumed line number of the current source line within the current source file. [Note 4: The presumed line number can be changed by the `#line` directive (15.8). — end note]

3143. Incorrect statement about enumerators for C23

Section: C.7.6 [diff.dcl] **Status:** ready **Submitter:** Hubert Tong **Date:** 2025-12-11

(From submission [#828](#).)

After C++ was rebased on C23, the statement in C.7.6 [diff.dcl] paragraph 12 that the type of enumerators in C is `int` is no longer correct (see WG14 N3029 and WG14 N3030).

Proposed resolution (approved by CWG 2026-03-27):

Change in C.7.6 [diff.dcl] paragraph 12 as follows:

Affected subclause: 9.8.1 [dcl.enum]

Change: In C++, the type of an enumerator is its enumeration. In C, the type of an enumerator is ~~int~~ **an integer type**.

[Example 10:

```
enum e { A };  
void f() {  
    auto x = A;  
    int *p = &x; // valid C, invalid C++  
}  
sizeof(A) == sizeof(int) // in C  
sizeof(A) == sizeof(e) // in C++  
/* and sizeof(int) is not necessarily equal to sizeof(e) */
```

—end example]

Rationale: In C++, an enumeration is a distinct type.

Effect on original feature: Change to semantics of well-defined feature.

Difficulty of converting: Semantic transformation.

How widely used: Seldom. ~~The only time this affects existing C code is when the size of an enumerator is taken. Taking the size of an enumerator is not a common C coding practice.~~

3145. Uniqueness of annotations

Section: 9.13.12 [dcl.attr.annotation] **Status:** ready **Submitter:** Barry Revzin **Date:** 2025-12-01

(From submission [#825](#).)

Consider:

```
template<int> int x [[=1]];  
template<int A> int y [[=A]];  
static_assert(annotations_of(^x<0>) == annotations_of(^x<1>)); // #1  
static_assert(annotations_of(^y<0>) == annotations_of(^y<1>)); // #2
```

Subclause 9.13.12 [dcl.attr.annotation] paragraph 3 specifies:

Each *annotation* produces a unique annotation.

It is unclear whether each instantiation of a template creates a distinct annotation.

Proposed resolution (approved by CWG 2026-03-27):

Change in 9.13.12 [dcl.attr.annotation] paragraph 3 and paragraph 4 as follows:

(3) Each *annotation* **or instantiation thereof** produces a unique annotation.

~~Substituting into an annotation is not in the immediate context.~~

[Example:

```
[[=1]] void f();
[[=2, =3, =2]] void g();
void g [[=4, =2]] ();
```

f has one annotation and g has five annotations. These can be queried with metafunctions such as `std::meta::annotations_of` (21.4.12 [\[meta.reflection.annotation\]](#)). —end example]

[Example:

```
template<int> int x [[=1]];
static_assert(annotations_of(^x<0>) != annotations_of(^x<1>)); // OK
-- end example]
```

(4) Substituting into an annotation is not in the immediate context.

[Example:

```
template<class T>
[[=T::type()]] void f(T t);
void f(int);
void g() {
    f(0); // OK
    f('0'); // error, ...
}
```

-- end example]

3148. Definition of "user-declared" special member function

Section: 11.4.1 [\[class.mem.general\]](#) **Status:** ready **Submitter:** Arthur O'Dwyer **Date:** 2026-01-12

(Split off from issue 2799.)

The standard does not define what a "user-declared" special member function is.

Proposed resolution (approved by CWG 2026-02-06):

1. Change in 9.10 [\[namespace.udecl\]](#) paragraph 4 as follows:

If a constructor or assignment operator brought from a base class into a derived class has the signature of a copy/move constructor or assignment operator for the derived class (11.4.5.3 [\[class.copy.ctor\]](#), 11.4.6 [\[class.copy.assign\]](#)), the *using-declaration* does not by itself suppress the implicit declaration of the derived class member; the member from the base class is hidden or overridden by the implicitly-declared copy/move constructor or assignment operator of the derived class, as described below. **[Note: A *using-declaration* that names a member function of a base class does not suppress the implicit declaration of a special member function in the derived class, even if their signatures are the same (11.4.5.2 [\[class.default.ctor\]](#), 11.4.5.3 [\[class.copy.ctor\]](#), 11.4.6 [\[class.copy.assign\]](#)). -- end note]**

2. Change in 11.4.1 [\[class.mem.general\]](#) paragraph 4 as follows:

... For any other *member-declaration*, each declared entity that is not an unnamed bit-field (11.4.10 [\[class.bit\]](#)) is a member of the class, and each such *member-declaration* shall either declare at least one member name of the class or declare at least one unnamed bit-field. **A user-declared entity is a direct member or a friend that, in either case, is declared by a member-declaration.**

3. Change in 11.4.5.2 [\[class.default.ctor\]](#) paragraph 1 as follows:

A default constructor for a class X is a constructor of class X for which each parameter that is not a function parameter pack has a default argument (including the case of a constructor with no parameters). If there is no **a class does not have a user-declared constructor or constructor template for class X**, a non-explicit constructor having no parameters is implicitly declared as defaulted (9.6 [\[dcl.fct.def\]](#)). An implicitly-declared default constructor is an inline public member of its class.

4. Change in 11.4.5.3 [\[class.copy.ctor\]](#) paragraph 6 as follows:

If the class **definition** does not **explicitly declare have a user-declared** copy constructor, a non-explicit one is declared implicitly. ...

5. Change in 11.4.5.3 [\[class.copy.ctor\]](#) paragraph 8 as follows:

If **the definition of** a class X does not **explicitly declare have a user-declared** move constructor, a non-explicit one will be implicitly declared as defaulted if and only if ...

6. Add a new paragraph before 11.4.5.3 [\[class.copy.ctor\]](#) paragraph 11 as follows:

[Note: A *using-declaration* in a derived class C that names a constructor from a base class never suppresses the implicit declaration of a copy/move constructor of C, even if the base class constructor would be a copy or move constructor if declared as a member of C. -- end note]

A copy/move constructor for class X is trivial if it is not user-provided and if: ...

7. Change in 11.4.6 [\[class.copy.assign\]](#) paragraph 2 as follows:

If the class **definition** does not **explicitly declare have a user-declared** copy assignment operator, one is declared implicitly. If the class **definition declares has a user-declared** move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defaulted (9.6 [\[dcl.fct.def\]](#)). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor (D.6 [\[depr.impldec\]](#)). ...

3149. Rvalues in destructuring expansion statements

Section: 8.7 [stmt.expand] **Status:** ready **Submitter:** Barry Revzin **Date:** 2026-01-14

(From submission [#835](#).)

Consider:

```
auto f() -> std::tuple<int, int&, int&&>;  
  
auto g() -> void {  
    template for (auto&& elem : f()) {  
        ;  
    }  
}
```

The type of `elem` is `int&` in each iteration, but the intent of [P1306R5](#) was that the first and third iteration yield `int&&` instead.

Proposed resolution (approved by CWG 2026-01-23):

Change in 8.7 [stmt.expand] bullet 5.3 as follows:

... where N is the structured binding size of the type of the *expansion-initializer* and S_i is

```
{  
    for-range-declaration = ++  $v_i$ ;  
    compound-statement  
}
```

If the *expansion-initializer* is an lvalue, then v_i is u_i ; otherwise, v_i is `static_cast<decltype( $u_i$ )&&>( $u_i$ )`.

3151. Closure types that are final

Section: 7.5.6.2 [expr.prim.lambda.closure] **Status:** ready **Submitter:** Jay Ghiron **Date:** 2025-12-25

(From submission [#832](#).)

The current specification leaves open the possibility that an implementation declares closure types as `final`. That is undesirable.

Proposed resolution (approved by CWG 2026-01-23):

Change in 7.5.6.2 [expr.prim.lambda.closure] paragraph 4 as follows:

The closure type is not an aggregate type (9.5.2 [dcl.init.aggr]) **and is not final (11.1 [class.pre])**; it is a structural type (13.2 [temp.param]) if and only if the lambda has no *lambda-capture*.

3153. Immediate-escalating defaulted comparison

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jason Merrill **Date:** 2026-01-25

(From submission [#837](#).)

Defaulted comparison operators should also be immediate-escalating.

Proposed resolution (approved by CWG 2026-02-06):

Change in 7.7 [expr.const] bullet 25.2 as follows:

An *immediate-escalating function* is

- the call operator of a lambda that is not declared with the `constexpr` specifier,
- a **non-user-provided** defaulted ~~special member~~ function that is not declared with the `constexpr` specifier, or
- a function that is not a prospective destructor and that results from the instantiation of a templated entity defined with the `constexpr` specifier.

...

3155. Escalation of virtual functions

Section: 11.7.3 [class.virtual] **Status:** ready **Submitter:** Cody Miller **Date:** 2026-02-06

Subclause 11.7.3 [class.virtual] paragraph 18 disallows mixing consteval and non-consteval virtual function overrides. This does not cover immediate functions via escalation.

Proposed resolution (approved by CWG 2026-02-20):

Change in 11.7.3 [class.virtual] paragraph 18 as follows:

A class with a ~~consteval~~ **an immediate** virtual function that overrides a **non-immediate** virtual function ~~that is not consteval~~ shall have consteval-only type (6.9.1 [basic.types.general]). ~~A consteval~~ **An immediate** virtual function shall not be overridden by a **non-immediate** virtual function ~~that is not consteval~~.

3156. Handling of deleted functions in unevaluated lambda-captures

Section: 7.5.6.3 [expr.prim.lambda.capture] **Status:** ready **Submitter:** Anoop S. Rana **Date:** 2026-01-29

(From submission #841.)

Consider:

```
struct C
{
    C(int) = delete;
    C(){};
};

decltype([b = C(3)]()){ return 4; }() x; //MSVC accepts; gcc and clang reject
```

Arguably, 7.5.6.3 [expr.prim.lambda.capture] paragraph 15 specifies that the initialization of the non-static data members only happens when the *lambda-expression* is evaluated, but there is no statement when the semantic constraints of such an initialization are checked.

Proposed resolution (approved by CWG 2026-03-27):

Change in 7.5.6.3 [expr.prim.lambda.capture] paragraph 15 as follows:

~~When the lambda-expression is evaluated, the~~ **The** entities that are captured by copy are used to direct-initialize each corresponding non-static data member of the resulting closure object, and the non-static data members corresponding to the *init-captures* are initialized as indicated by the corresponding initializer (which may be copy- or direct-initialization). (For array members, the array elements are direct-initialized in increasing subscript order.) These initializations are performed **when the lambda-expression is evaluated and** in the (unspecified) order in which the non-static data members are declared.

3157. Missing handling of operator new[] for deallocation function template matching

Section: 13.10.3.7 [temp.deduct.decl] **Status:** ready **Submitter:** Jiang An **Date:** 2026-01-26

(From submission #838.)

Subclause 13.10.3.7 [temp.deduct.decl] paragraph 1 does not handle array-new.

Proposed resolution (approved by CWG 2026-02-20):

1. Change in 13.10.3.7 [temp.deduct.decl] paragraph 1 as follows:

In a declaration whose *declarator-id* refers to a specialization of a function template, template argument deduction is performed to identify the specialization to which the declaration refers. Specifically, this is done for explicit instantiations (13.9.3 [temp.explicit]), explicit specializations (13.9.4 [temp.expl.spec]), and certain friend declarations (13.7.5 [temp.friend]). This is also done to determine whether a deallocation function template specialization matches a placement ~~operator new~~ **allocation function** (6.8.6.5.3 [basic.stc.dynamic.deallocation], 7.6.2.8 [expr.new]). In all these cases, P is the type of the function template being considered as a potential match and A is either the function type from the declaration or the type of the deallocation function that would match the placement ~~operator new~~ **allocation function** as described in 7.6.2.8 [expr.new]. The deduction is done as described in 13.10.3.6 [temp.deduct.type].

2. Change in 13.7.7.3 [temp.func.order] bullet 1.3 as follows:

- ...
- when a placement ~~operator delete~~ **deallocation function** that is a function template specialization is selected to match a placement ~~operator new~~ **allocation function** (6.8.6.5.3 [basic.stc.dynamic.deallocation], 7.6.2.8 [expr.new]);
- ...

3162. Evaluation context of manifestly constant-evaluated expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Dan Katz **Date:** 2026-02-13

(From submission #848.)

Consider:

```

struct S;
constexpr size_t f() {
    constexpr bool b = is_complete_type(^S);
    return b ? 1 : 2;
}

struct T;
constexpr {
    define_aggregate(^S, {});
    define_aggregate(^T, {
        data_member_spec(^int, {.name="m", .bit_width=f()}),
    });
}

int main() {
    return bit_size_of(^T::m);
}

```

The Working Draft arguably says that this program should exit with status 1: During the evaluation of the expression corresponding to the `constexpr`-block, the evaluation of `is_complete_type(^S)` contains the synthesized point associated with the injected declaration of `S`; therefore, the type is complete.

But this was never the intention of P2996; the intention was to evaluate manifestly constant-evaluated expressions at the point where they appear. The problem is that we didn't consider manifestly constant-evaluated expressions that might be executed (formally, per the abstract machine) during the evaluation of other manifestly constant-evaluated expressions.

Proposed resolution (approved by CWG 2026-03-25):

This also resolves issue 3127.

Change in 7.7 [expr.const] paragraph 30 as follows:

During an evaluation V (6.10.1 [intro.execution]) of an expression, conversion, or initialization E as a core constant expression, the *point of evaluation of E during V* is the program point P determined as follows:

- If E is a potentially-evaluated subexpression of a default member initializer I , and V is the evaluation of E in the evaluation of I as an immediate subexpression of a (possibly aggregate) initialization, then P is the point of evaluation of that initialization. [Note: For example, E can be an immediate invocation in a default member initializer used by an aggregate initialization appearing at P . -- end note]
- Otherwise, if E is a potentially-evaluated subexpression of a default argument A (9.3.4.7 [dcl.fct.default]), and V is the evaluation of E in the evaluation of A as an immediate subexpression of a function call (7.6.1.3 [expr.call]), then P is the point of evaluation of that function call.
- Otherwise, P is the point at which E appears.

The *evaluation context* is a set of program points that determines the behavior of certain functions used for reflection (21.4 [meta.reflection]). During the evaluation V of an expression E as a core constant expression, the *evaluation context* of an evaluation X (6.10.1 [intro.execution]) consists of the following points during V is the set C of program points determined as follows:

- The program point $EVAL_PT(L)$, where L is the point at which E appears, and where $EVAL_PT(P)$, for a point P , is a point R determined as follows:
 - If a potentially-evaluated subexpression (6.10.1 [intro.execution]) of a default member initializer I appears at P , and a (possibly aggregate) initialization during V is using I , then R is $EVAL_PT(Q)$ where Q is the point at which that initialization appears.
 - Otherwise, if a potentially-evaluated subexpression of a default argument (9.3.4.7 [dcl.fct.default]) appears at P , and an invocation of a function (7.6.1.3 [expr.call]) during V is using that default argument, then R is $EVAL_PT(Q)$ where Q is the point at which that invocation appears.
 - Otherwise, R is P .
- If X occurs during the evaluation Y of a manifestly constant-evaluated expression, where Y occurs during V , then C is the evaluation context of X during Y .
- Otherwise, C contains
 - the point of evaluation of E during V and each synthesized point in the instantiation context thereof and
 - Each synthesized point corresponding to an injected declaration produced by any evaluation executed during V that is sequenced before X (6.10.1 [intro.execution]).

[Note: The evaluation context determines the behavior of certain functions used for reflection (21.4 [meta.reflection]). -- end note]

[Example:

```

struct S;
constexpr std::size_t f(int p) {
    constexpr std::size_t r = /* Q */ std::meta::is_complete_type(^S) ? 1 : 2; // #1
    if (!std::meta::is_complete_type(^S)) { // #2
        std::meta::define_aggregate(^S, {});
    }
    return (p > 0) ? f(p - 1) : r;
}

constexpr {
    if (f(1) != 2) {
        throw; // OK, not evaluated
    }
}

```

During each evaluation of `std::meta::is_complete_type(^S)` at #1 (21.4.7 [meta.reflection.queries]) that is executed during the evaluation of `f(1) != 2`, the evaluation context contains `Q`, but does not contain the synthesized point associated with the injected declaration of `S`. However, the synthesized point is in the evaluation context of `std::meta::is_complete_type(^S)` at #2 during the evaluation of `f(0)`. -- end example]

All C++ implementations have converged on "strong ownership" of modules, with none planning to implement "weak ownership". This disposition no longer serves any purpose other than removing certainty.

Proposed resolution (approved by CWG 2026-03-26):

1. Change in 6.7 [basic.link] paragraph 8 as follows:

Two declarations of entities declare the same entity if, considering declarations of unnamed types to introduce their names for linkage purposes, if any (9.2.4 [dcl.typedef], 9.8.1 [dcl.enum]), they correspond (6.4.1 [basic.scope.scope]), have the same target scope that is not a function or template parameter scope, neither is a name-independent declaration, and either

- they appear in the same translation unit, or
- they both declare type aliases or namespace aliases that have the same underlying entity, or
- they both declare names with module **or external** linkage and are attached to the same module; **or**,
- ~~they both declare names with external linkage.~~

[Note 3: There are other circumstances in which declarations declare the same entity (9.12 [dcl.link], 13.6 [temp.type], 13.7.6 [temp.spec.partial]). — end note]

2. Change in 6.7 [basic.link] paragraph 10 as follows:

[Note: If two declarations of an entity **correspond but** are attached to different modules, the program is ill-formed; **no diagnostic is required** if **neither is reachable from one precedes** the other (6.4.1 [basic.scope.scope]) **-- end note**]. [Example 2:

```
"decls.h":
    int f();    // #1, attached to the global module
    int g();    // #2, attached to the global module
```

Module interface of M:

```
module;
#include "decls.h"
export module M;
export using ::f;    // OK, does not declare an entity, exports #1
int g();             // error: matches corresponds to #2, but attached to M
export int h();      // #3
export int k();      // #4
```

Other translation unit:

```
import M;
static int h();      // error: matches conflicts with #3
int k();             // error: matches conflicts with #4
```

3172. Reference to wrong placeholder

Section: 13.5.4 [temp.constr.normal] Status: ready Submitter: Corentin Jabot Date: 2025-04-02

(From submission #696.)

(Split off from issue 3069.)

Subclause 13.5.4 [temp.constr.normal] bullet 1.4.2 specifies:

... The normal form of CI is the result of substituting, in the normal form N of CE, appearances of C's template parameters in the parameter mappings of the atomic constraints in N with their respective arguments from C. ...

However, C is a *concept-name* and does not have any template parameters. CI, the concept-id, does, however.

Proposed resolution (approved by CWG 2026-03-27):

Change in 13.5.4 [temp.constr.normal] paragraph 1.4.2:

For a concept-id C<A1 , A2 , . . . , An > termed CI:

- If C names a dependent concept, the normal form of CI is a concept-dependent constraint whose *concept-id* is CI and whose parameter mapping is the identity mapping.
- Otherwise, to form CE, any non-dependent concept template argument Ai is substituted into the *constraint-expression* of C. If any such substitution results in an invalid concept-id, the program is ill-formed; no diagnostic is required. The normal form of CI is the result of substituting, in the normal form N of CE, appearances of C's template parameters in the parameter mappings of the atomic constraints in N with their respective arguments from **e ci**. If any such substitution results in an invalid type or expression, the program is ill-formed; no diagnostic is required.

3173. Remove misleading footnote about as-if rule

Section: 6.8.2 [intro.object] Status: ready Submitter: CWG Date: 2026-03-27

This footnote in 6.8.2 [\[intro.object\]](#) paragraph 10, by its mere presence, gives the misguided impression that the as-if rule applies to select parts of C++ only:

Under the “as-if” rule an implementation is allowed to store two objects at the same machine address or not store an object at all if the program cannot observe the difference (6.10.1 [\[intro.execution\]](#)).

Proposed resolution (approved by CWG 2027-03-27) :

Remove footnote 16 in 6.8.2 [\[intro.object\]](#) paragraph 10 as follows:

~~Under the “as-if” rule an implementation is allowed to store two objects at the same machine address or not store an object at all if the program cannot observe the difference (6.10.1 [\[intro.execution\]](#)).~~